What is Amazon Elastic Container Service?

Amazon Elastic Container Service

Amazon Elastic Container Service (Amazon ECS) is a highly scalable, fast container management service that makes it easy to run, stop, and manage containers on a cluster. Your containers are defined in a task definition that you use to run individual tasks or tasks within a service. In this context, a service is a configuration that enables you to run and maintain a specified number of tasks simultaneously in a cluster. You can run your tasks and services on a serverless infrastructure that is managed by AWS Fargate. Alternatively, for more control over your infrastructure, you can run your tasks and services on a cluster of Amazon EC2 instances that you manage.

Amazon ECS enables you to launch and stop your container-based applications by using simple API calls. You can also retrieve the state of your cluster from a centralized service and have access to many familiar Amazon EC2 features.

Amazon Elastic Container Service

You can schedule the placement of your containers across your cluster based on your resource needs, isolation policies, and availability requirements. With Amazon ECS, you don't have to operate your own cluster management and configuration management systems or worry about scaling your management infrastructure.

Amazon ECS can be used to create a consistent build and deployment experience, to manage and scale batch and Extract-Transform-Load (ETL) workloads, and to build sophisticated application architectures on a microservices model.

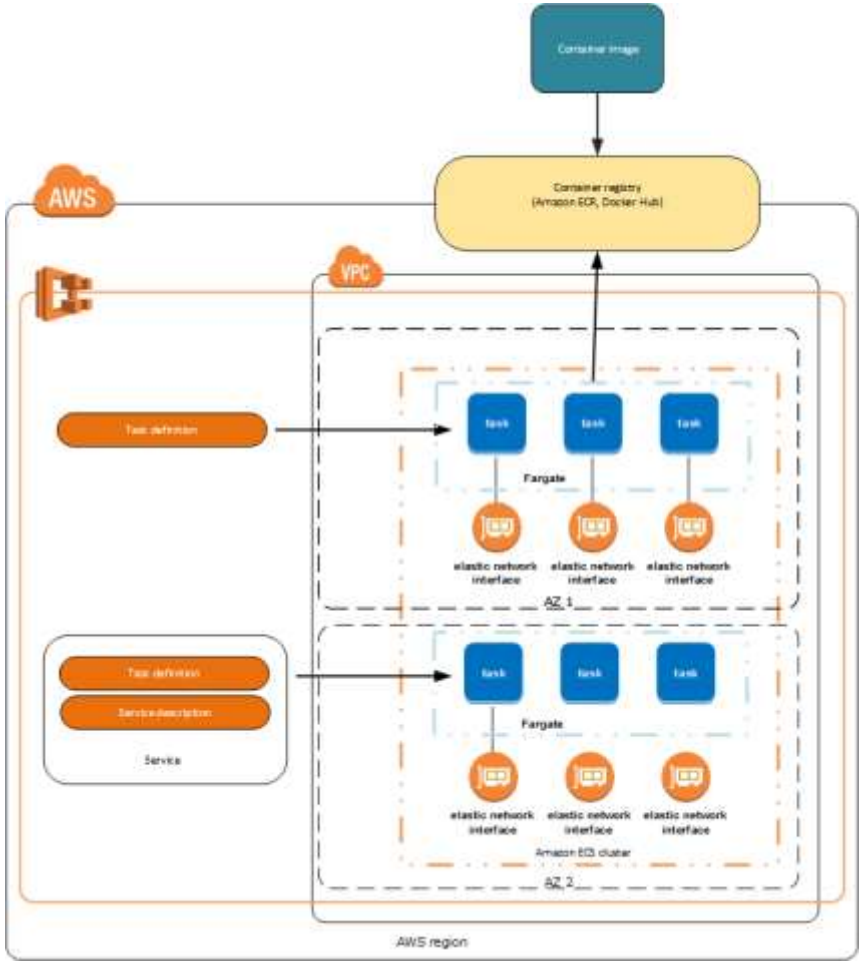
Features of Amazon ECS

Amazon ECS is a regional service that simplifies running containers in a highly available manner across multiple Availability Zones within a Region. You can create Amazon ECS clusters within a new or existing VPC. After a cluster is up and running, you can create task definitions that define which container images run across your clusters. Your task definitions are used to run tasks or create services. Container images are stored in and pulled from container registries, for example, the Amazon Elastic Container Registry.

The following diagram shows the architecture of an Amazon ECS environment run on AWS Fargate.

Features of Amazon ECS

The following sections dive into these individual elements of the Amazon ECS architecture in more detail.

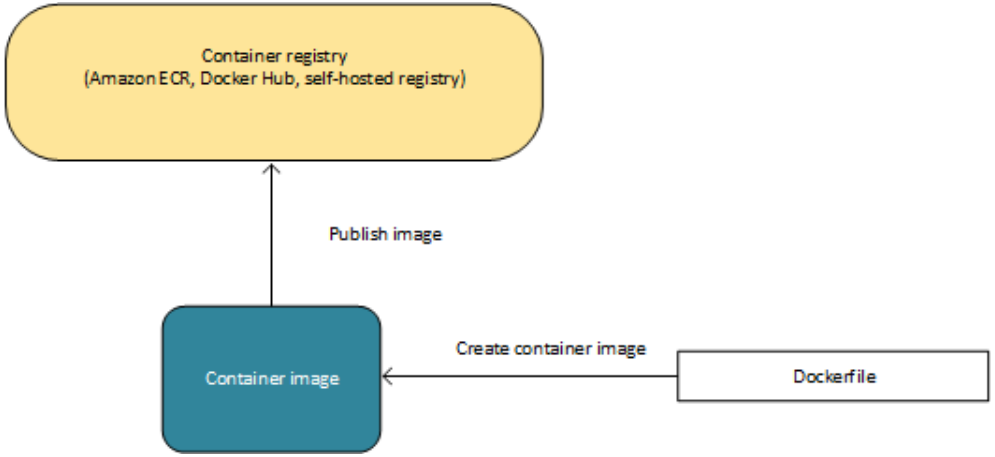


Containers and images

To deploy applications on Amazon ECS, your application components must be architected to run in containers. A container is a standardized unit of software development that contains everything that your software application needs to run, including relevant code, runtime, system tools, and system libraries.

Containers are created from a read-only template called an image.

Images are typically built from a Dockerfile, which is a plaintext file that specifies all of the components that are included in the container. After being built, these images are stored in a registry where they then can be downloaded and run on your cluster.



Task definitions

To prepare your application to run on Amazon ECS, you must create a task definition. The task definition is a text file (in JSON format) that describes one or more containers (up to a maximum of ten) that form your application. The task definition can be thought of as a blueprint for your application. It specifies various parameters for your application. For example, these parameters can be used to indicate which containers should be used, which ports should be opened for your application, and what data volumes should be used with the containers in the task. The specific parameters available for your task definition depend on the needs of your specific application.

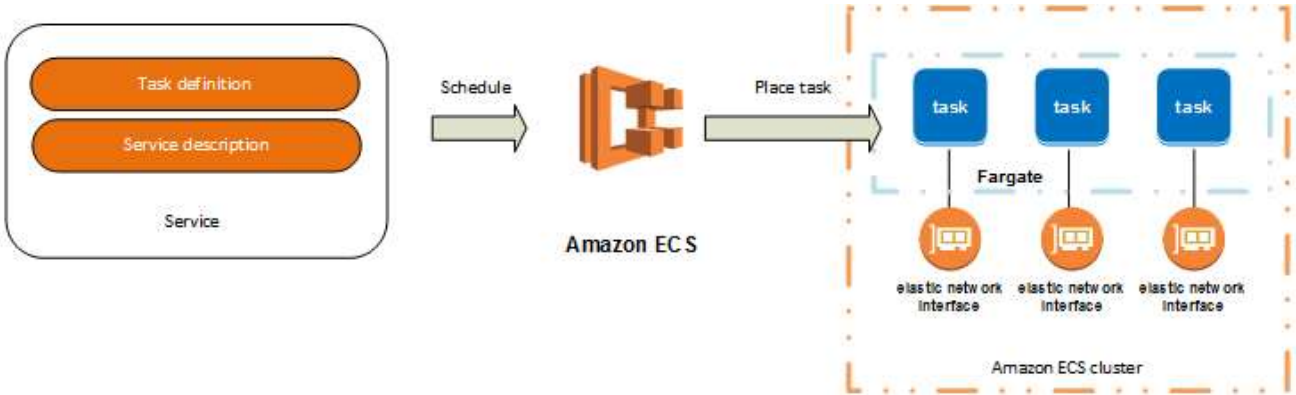
The following is an example of a task definition that specifies the use of Fargate to launch a single container that runs an NGINX web server.

```
{
    "family": "webserver",
    "containerDefinitions": [
        {
            "name": "web",
            "image": "nginx",
            "memory": "100",
            "cpu": "99"
        },
    ],
    "requiresCompatibilities": [
        "FARGATE"
    ],
    "networkMode": "awsvpc",
    "memory": "512",
    "cpu": "256",
}
```

Tasks and scheduling

A task is the instantiation of a task definition within a cluster. After you have created a task definition for your application within Amazon ECS, you can specify the number of tasks to run on your cluster.

The Amazon ECS task scheduler is responsible for placing tasks within your cluster. There are several different scheduling options available. For example, you can define a service that runs and maintains a specified number of tasks simultaneously.



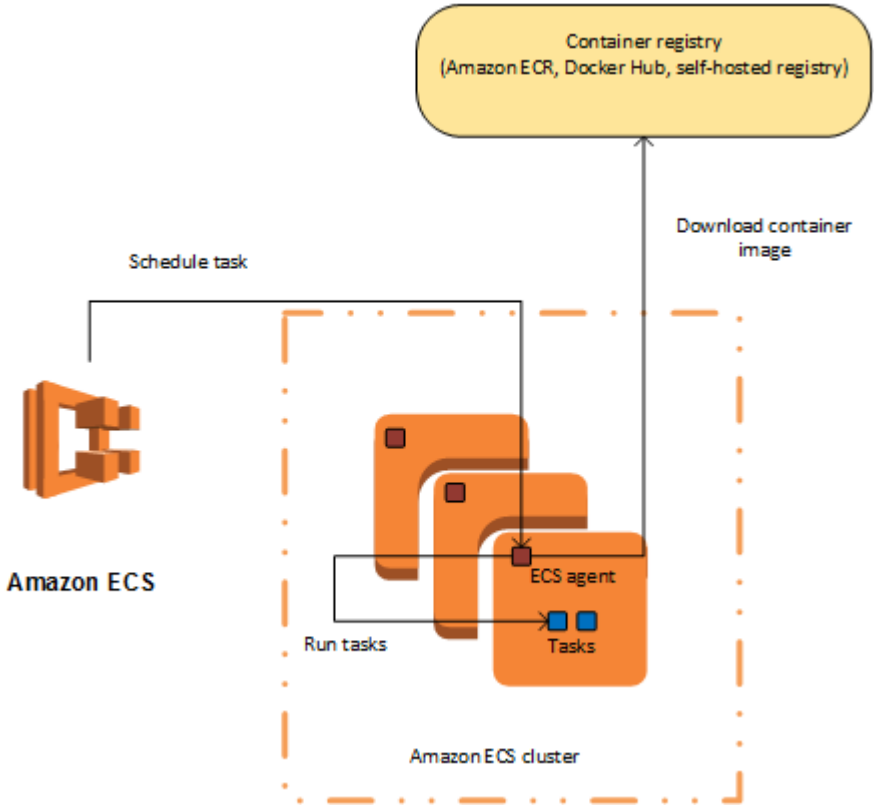
Clusters

An Amazon ECS cluster is a logical grouping of tasks or services. You can register one or more Amazon EC2 instances (also referred to as container instances) with your cluster to run tasks on them. Or, you can use the serverless infrastructure that Fargate provides to run tasks. When your tasks are run on Fargate, your cluster resources are also managed by Fargate.

When you first use Amazon ECS, a default cluster is created for you. You can create additional clusters in an account to keep your resources separate.

Container agent

The container agent runs on each container instance within an Amazon ECS cluster. The agent sends information about the resource's current running tasks and resource utilization to Amazon ECS. It starts and stops tasks whenever it receives a request from Amazon ECS.



Getting started with Amazon ECS

Getting started with Amazon ECS

Amazon Elastic Container Service (Amazon ECS) is a highly scalable, fast, container management service that makes it easy to run, stop, and manage your containers. You can host your containers on a serverless infrastructure that is managed by Amazon ECS by launching your services or tasks using the Fargate launch type. For more control, you can host your tasks on a cluster of Amazon EC2 instances that you manage by using the EC2 launch type.

Step 1: Register a task definition

A task definition is like a blueprint for your application. Each time that you launch a task in Amazon ECS, you specify a task definition. The service then knows which Docker image to use for containers, how many containers to use in the task, and the resource allocation for each container.

The following steps walk you through creating a task definition that will deploy a simple web application.

To register a task definition

1. Open the Amazon ECS console at https://console.aws.amazon.com/ecs/.
2. From the navigation bar, select the Region you want to use.
3. In the navigation pane, choose **Task Definitions, Create new Task Definition**.
4. On the **Select launch type compatibility** page, select **EC2** and choose **Next step**.

Step 1: Register a task definition

5. On the **Configure task and container definitions** page, scroll down and choose **Configure via JSON**.
6. Copy and paste the following example task definition into the box and then choose **Save**.
7. **Choose Create.**

```
{
    "containerDefinitions": [
        {
            "entryPoint": [
                "sh",
                "-c"
            ],
            "portMappings": [
                {
                    "hostPort": 80,
                    "protocol": "tcp",
                    "containerPort": 80
                }
            ],
            "command": [
                "/bin/sh -c \"echo '<html> <head> <title>Amazon ECS Sample App</title>
            ],
            "cpu": 10,
            "memory": 300,
            "image": "httpd:2.4",
            "name": "simple-app"
        }
    ],
    "family": "console-sample-app-static"
}
```

Step 2: Create a cluster

An Amazon ECS cluster is a logical grouping of tasks, services, and container instances. When creating a cluster using the console, Amazon ECS creates a AWS CloudFormation stack that takes care of the Amazon EC2 instance creation, networking and IAM configuration for you.

The following steps walk you through creating a cluster with one Amazon EC2 instance registered to it which will enable us to run a task on it. If a specific field is not mentioned, leave the default value the console uses.

To create a cluster

1. Open the Amazon ECS console at https://console.aws.amazon.com/ecs/.
2. From the navigation bar, select the same Region you used in the previous step.
3. In the navigation pane, choose **Clusters**.

Step 2: Create a cluster

4. On the **Clusters** page, choose **Create Cluster**.
5. On the **Select cluster template** page, choose **EC2 Linux + Networking**.
6. For **Cluster name**, choose a name for your cluster.
7. In the **Instance configuration** section, do the following:
    a. For **EC2 instance type**, choose either the **t2.micro** or **t3.micro** instance type to use for the container instance. Instance types with more CPU and memory resources can handle more tasks, but that is unnecessary for this getting started guide. For more information about the different instance types, see Amazon EC2 Instances.
    b. For **Number of instances**, type **1**. Amazon EC2 instances incur costs while they exist in your AWS resources. For more information, see Amazon EC2 Pricing.
    c. For **EC2 Ami Id**, use the default value which is the Amazon Linux 2 Amazon ECS-optimized AMI.

Step 2: Create a cluster

8. In the **Networking** section, for **VPC** choose either **Create a new VPC** to have Amazon ECS create a new VPC for the cluster to use, or choose an existing VPC to use.
9. In the **Container instance IAM role** section, choose **Create new role** to have Amazon ECS create a new IAM role for your container instances, or choose an existing Amazon ECS container instance (ecsInstanceRole) role that you have already created.
10. Choose **Create**.

Step 3: Create a Service

An Amazon ECS service enables you to run and maintain a specified number of instances of a task definition simultaneously in an Amazon ECS cluster. If any of your tasks should fail or stop for any reason, the Amazon ECS service scheduler launches another instance of your task definition to replace it in order to maintain the desired number of tasks in the service.

To create a service

1. Open the Amazon ECS console at https://console.aws.amazon.com/ecs/.
2. From the navigation bar, select the same Region you used in the previous step.
3. In the navigation pane, choose **Clusters**.
4. Select the cluster you created in the previous step.
5. On the **Services** tab, choose **Create**.

Step 3: Create a Service

6. In the **Configure service** section, do the following:
    a. For **Launch type**, select **EC2**
    b. For **Task definition**, select the **console-sample-app-static** task definition you created in step 1.
    c. For **Cluster**, select the cluster you created in step 2.
    d. For **Service name**, select a name for your service.
    e. For **Number of tasks**, enter 1.
7. Use the default values for the rest of the fields and choose **Next step**.
8. In the **Configure network** section, leave the default values and choose **Next step**.
9. In the **Set Auto Scaling** section, leave the default value and choose **Next step**.
10. Review the options and choose **Create service**.
11. Choose **View service** to review your service.

Step 4: View your Service

The service is a web-based application so you can view its containers with a web browser.

To view the service details

1. Open the Amazon ECS console at https://console.aws.amazon.com/ecs/.
2. From the navigation bar, select the same Region you used in the previous step.
3. In the navigation pane, choose **Clusters**.
4. Select the cluster you created step 2.
5. On the **Services** tab, choose the service you created in step 3.
6. On the **Service:** *service-name* page, choose the **Tasks** tab.
7. Confirm that the task is in a **RUNNING** state. If it is, select the task to view the task details. If it is not in a **RUNNING** status, refresh the service details screen until it is.

Step 4: View your Service

8. In the **Containers** section, expand the container details. In the **Network bindings** section, for **External Link** you will see the **IPv4 Public IP** address to use to access the web application.

9. Enter the **IPv4 Public IP** address in your web browser and you should see a webpage that displays the **Amazon ECS sample** application.

Step 5: Clean Up

When you are finished using an Amazon ECS cluster, you can clean up the resources associated with it to avoid incurring charges for resources that you are not using. The Amazon ECS resources created in this getting started guide, such as the cluster and service can be cleaned up using the Amazon ECS console.

To clean up the resources

1. Open the Amazon ECS console at https://console.aws.amazon.com/ecs/.
2. In the navigation pane, choose **Clusters**.
3. Select the cluster you created step 2.
4. On the **Services** tab, select the service you created in step 3 and choose **Delete**. At the confirmation prompt, enter **delete me** and then choose **Delete**.
5. On the cluster details page, choose **Delete cluster**. At the confirmation prompt, enter **delete me** and then choose **Delete**. Deleting the cluster cleans up the associated resources that were created with the cluster, including the VPC and Amazon EC2 instances.

Getting started with Amazon ECS using Fargate

Step 1: Create a Task Definition

A task definition is like a blueprint for your application. Each time you launch a task in Amazon ECS, you specify a task definition. The service then knows which Docker image to use for containers, how many containers to use in the task, and the resource allocation for each container.

1. Open the Amazon ECS console first-run wizard at https://console.aws.amazon.com/ecs/home#/firstRun.
2. From the navigation bar, select the **US East (N. Virginia)** Region.
3. Configure your container definition parameters.

   For **Container definition**, the first-run wizard comes preloaded with the sample-app, nginx, and tomcat-webserver container definitions in the console. You can optionally rename the container or review and edit the resources used by the container (such as CPU units and memory limits) by choosing **Edit** and editing the values shown. For more information, see Container Definitions.

Step 1: Create a Task Definition

5. For **Task definition**, the first-run wizard defines a task definition to use with the preloaded container definitions. You can optionally rename the task definition and edit the resources used by the task (such as the **Task memory** and **Task CPU** values) by choosing **Edit** and editing the values shown. Task definitions created in the first-run wizard are limited to a single container for simplicity. You can create multi-container task definitions later in the Amazon ECS console.
6. Choose **Next**.

Step 2: Configure the Service

In this section of the wizard, select how to configure the Amazon ECS service that is created from your task definition. A service launches and maintains a specified number of copies of the task definition in your cluster. The **Amazon ECS sample** application is a web-based Hello World–style application that is meant to run indefinitely. By running it as a service, it restarts if the task becomes unhealthy or unexpectedly stops.

The first-run wizard comes preloaded with a service definition, and you can see the sample-app-service service defined in the console. You can optionally rename the service or review and edit the details by choosing **Edit** and doing the following:

1. In the **Service name** field, select a name for your service.
2. In the **Number of desired tasks** field, enter the number of tasks to launch with your specified task definition.

Step 2: Configure the Service

3. In the **Security group** field, specify a range of IPv4 addresses to allow inbound traffic from, in CIDR block notation. For example, 203.0.113.0/24.
4. (Optional) You can choose to use an Application Load Balancer with your service. When a task is launched from a service that is configured to use a load balancer, the task is registered with the load balancer. Traffic from the load balancer is distributed across the instances in the load balancer. Complete the following steps to use a load balancer with your service.
    a. In the **Container to load balance** section, choose the **Load balancer listener port**. The default value here is set up for the sample application, but you can configure different listener options for the load balancer.
5. Review your service settings and click **Save**, **Next**.

Step 3: Configure the Cluster

In this section of the wizard, you name your cluster, and then Amazon ECS takes care of the networking and IAM configuration for you.

1. In the **Cluster name** field, choose a name for your cluster.
2. Click **Next** to proceed.

Step 4: Review

1. Review your task definition, task configuration, and cluster configuration and click **Create** to finish. You are directed to a **Launch Status** page that shows the status of your launch. It describes each step of the process (this can take a few minutes to complete while your Auto Scaling group is created and populated).
2. After the launch is complete, choose **View service**.

Step 5: View your Service

If your service is a web-based application, such as the **Amazon ECS sample** application, you can view its containers with a web browser.

1. On the **Service:** *service-name* page, choose the **Tasks** tab.
2. Choose a task from the list of tasks in your service.
3. In the **Network** section, choose the **ENI Id** for your task. This takes you to the Amazon EC2 console where you can view the details of the network interface associated with your task, including the **IPv4 Public IP** address.
4. Enter the **IPv4 Public IP** address in your web browser and you should see a webpage that displays the **Amazon ECS sample** application.

Step 6: Clean Up

When you are finished using an Amazon ECS cluster, you should clean up the resources associated with it to avoid incurring charges for resources that you are not using. Some Amazon ECS resources, such as tasks, services, clusters, and container instances, are cleaned up using the Amazon ECS console. Other resources, such as Amazon EC2 instances, Elastic Load Balancing load balancers, and Auto Scaling groups, must be cleaned up manually in the Amazon EC2 console or by deleting the AWS CloudFormation stack that created them.

1. Open the Amazon ECS console at https://console.aws.amazon.com/ecs/.
2. In the navigation pane, choose **Clusters**.
3. On the **Clusters** page, select the cluster to delete.
4. Choose **Delete Cluster**. At the confirmation prompt, enter **delete me** and then choose **Delete**. Deleting the cluster cleans up the associated resources that were created with the cluster, including Auto Scaling groups, VPCs, or load balancers.

Q&A